

**LAPORAN**  
**TUGAS SISTEM OPERASI MAIN MEMORY**



Dosen pengampu:  
**Triawan Adi Cahyanto M.Kom**

Dibuat Oleh:  
**Nadianur Anisa (2410651026)**

**PRODI TEKNIK INFORMATIKA**  
**FAKULTAS TEKNIK**  
**UNIVERSITAS MUHAMMADIYAH JEMBER**  
**2025**

## TUGAS 1: ANALISIS MEMORY LAYOUT

### A. MEMODIFIKASI PROGRAM

File: tugas1\_address.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

// Global & static data
int global_var = 100;
int uninitialized_var;
static int static_array[5] = {1,2,3,4,5};

// String literal
char *string_literal = "Sistem Operasi";

// Function declarations
void func1();
void func2();
void recursive_func(int level);

void func1() {
    int local = 10;
    printf("func1() address      : %p\n", (void*)func1);
    printf("func1 local var addr : %p\n", (void*)&local);
}

void func2() {
    int local = 20;
    printf("func2() address      : %p\n", (void*)func2);
    printf("func2 local var addr : %p\n", (void*)&local);
}

void recursive_func(int level) {
    int recursive_var = level;
    printf("Recursive level %d - stack addr: %p\n",
        level, (void*)&recursive_var);

    if (level < 3) {
        recursive_func(level + 1);
    }
}

int main() {
    int local_main = 50;
```

```

int *dynamic_array = (int*)malloc(5 * sizeof(int));

printf("=== ENHANCED MEMORY LAYOUT ===\n\n");

// Text Segment
printf("1. TEXT SEGMENT\n");
printf("main() address      : %p\n", (void*)main);
printf("func1() address       : %p\n", (void*)func1);
printf("func2() address       : %p\n\n", (void*)func2);

// Data Segment
printf("2. DATA SEGMENT\n");
printf("global_var address   : %p\n", (void*)&global_var);
printf("static_array address  : %p\n", (void*)&static_array[0]);
printf("string_literal addr   : %p\n\n", (void*)string_literal);

// Heap
printf("3. HEAP SEGMENT\n");
printf("dynamic_array addr    : %p\n\n", (void*)dynamic_array);

// Stack
printf("4. STACK SEGMENT\n");
printf("main local var addr   : %p\n\n", (void*)&local_main);

printf("5. RECURSIVE STACK\n");
recursive_func(0);

printf("\nPID: %d\n", getpid());
printf("Tekan ENTER untuk melihat /proc/[pid]/maps\n");
getchar();

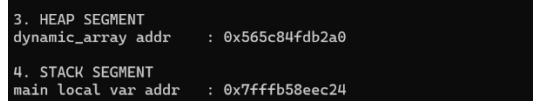
char cmd[100];
sprintf(cmd, "cat /proc/%d/maps", getpid());
system(cmd);

free(dynamic_array);
return 0;
}

```

## B. ANALISIS

1. Berapa jarak (dalam bytes) antara:
  - Stack dan Heap?



```

3. HEAP SEGMENT
dynamic_array addr    : 0x565c84fdb2a0

4. STACK SEGMENT
main local var addr   : 0x7fffb58eec24

```

- Text segment dan Data segment?

```
1. TEXT SEGMENT
main() address      : 0x565c7b908398
func1() address     : 0x565c7b908249
func2() address     : 0x565c7b9082bb

2. DATA SEGMENT
global_var address  : 0x565c7b90b010
static_array address : 0x565c7b90b020
string_literal addr  : 0x565c7b909008
```

- Alamat fungsi satu dengan lainnya?

```
func1() address      : 0x565c7b908249
func2() address      : 0x565c7b9082bb
```

2. Jalankan program 5 kali, catat alamat-alamat yang muncul

- Alamat mana yang selalu sama?

Urutan segment (text → data → heap → stack)

Jarak relatif antar variabel dalam segment yang sama

- Alamat mana yang berubah?

Alamat fungsi, Heap, Stack

- Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

Disebabkan oleh ASLR (Address Space Layout Randomization) yaitu Sistem operasi mengacak alamat memori, Meningkatkan keamanan, Mencegah buffer overflow & memory exploit

3. Bandingkan dengan memory map dari /proc/[pid]/maps:

- Apakah alamat yang Anda dapatkan sesuai dengan range di memory map?

Ya, Alamat sesuai dengan memory map

- Sebutkan permission (rwx) dari setiap segment

| Segment | Permission |
|---------|------------|
| Text    | r-xp       |
| Data    | rw-p       |

|       |      |
|-------|------|
| Heap  | rw-p |
| Stack | rw-p |

## C. HASIL OUTPUT

- Hasil 1x dijalankan

```

cd /praktikum-memory
dea@LAPTOP-H54VQ765:~/praktikum-memory$ nano tugas1_address.c
dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas1_address.c -o tugas1_address
./tugas1_address
=== ENHANCED MEMORY LAYOUT ===

1. TEXT SEGMENT
main() address      : 0x565c7b908398
func1() address     : 0x565c7b908249
func2() address     : 0x565c7b9082bb

2. DATA SEGMENT
global_var address  : 0x565c7b90b010
static_array address : 0x565c7b90b020
string_literal addr  : 0x565c7b909008

3. HEAP SEGMENT
dynamic_array addr   : 0x565c84fdb2a0

4. STACK SEGMENT
main local var addr  : 0x7ffffb58eec24

5. RECURSIVE STACK
Recursive level 0 - stack addr: 0x7ffffb58eec04
Recursive level 1 - stack addr: 0x7ffffb58eebd4
Recursive level 2 - stack addr: 0x7ffffb58eeba4
Recursive level 3 - stack addr: 0x7ffffb58eeb74

PID: 772
Tekan ENTER untuk melihat /proc/[pid]/maps

565c7b907000-565c7b908000 r--p 00000000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
565c7b908000-565c7b909000 r-xp 00001000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
565c7b909000-565c7b90a000 r--p 00002000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
565c7b90a000-565c7b90b000 r--p 00003000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
565c7b90b000-565c7b90c000 rw-p 00004000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
565c84fdb000-565c84ffc000 rw-p 00000000 00:00 0 [heap]
7ad309a00000-7ad309a28000 r--p 00000000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ad309a28000-7ad309abb000 r-xp 00028000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ad309abb000-7ad309bff000 r--p 001b0000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ad309bff000-7ad309c03000 r--p 001fe000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ad309c03000-7ad309c05000 rw-p 00202000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ad309c05000-7ad309c12000 rw-p 00000000 00:00 0
7ad309d96000-7ad309d99000 rw-p 00000000 00:00 0
7ad309da0000-7ad309da2000 rw-p 00000000 00:00 0
7ad309da2000-7ad309da3000 r--p 00000000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ad309da3000-7ad309dce000 r-xp 00001000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ad309dce000-7ad309dd8000 r--p 0002c000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ad309dd8000-7ad309dda000 r--p 00036000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ad309dda000-7ad309ddc000 rw-p 00038000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffffb58d0000-7ffffb58f1000 rw-p 00000000 00:00 0 [stack]
7ffffb5990000-7ffffb599d000 r--p 00000000 00:00 0 [vvar]
7ffffb599d000-7ffffb599f000 r-xp 00000000 00:00 0 [vdso]

```

- Hasil ke 2x dijalankan

```

dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas1_address.c -o tugas1_address
./tugas1_address
=== ENHANCED MEMORY LAYOUT ===

1. TEXT SEGMENT
main() address      : 0x593e07092398
func1() address     : 0x593e07092249
func2() address     : 0x593e070922bb

2. DATA SEGMENT
global_var address  : 0x593e07095010
static_array address : 0x593e07095020
string_literal addr  : 0x593e07093008

3. HEAP SEGMENT
dynamic_array addr   : 0x593e4483f2a0

4. STACK SEGMENT
main local var addr  : 0x7ffcd8d748c4

5. RECURSIVE STACK
Recursive level 0 - stack addr: 0x7ffcd8d748a4
Recursive level 1 - stack addr: 0x7ffcd8d74874
Recursive level 2 - stack addr: 0x7ffcd8d74844
Recursive level 3 - stack addr: 0x7ffcd8d74814

PID: 784
Tekan ENTER untuk melihat /proc/[pid]/maps

593e07091000-593e07092000 r--p 00000000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
593e07092000-593e07093000 r-xp 00001000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
593e07093000-593e07094000 r--p 00002000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
593e07094000-593e07095000 r--p 00003000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
593e07095000-593e07096000 rw-p 00003000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
593e4483f000-593e44860000 rw-p 00000000 00:00 0 [heap]
76889a800000-76889a820000 r--p 00000000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
76889a820000-76889a9b0000 r-xp 00028000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
76889a9b0000-76889a9ff000 r--p 001b0000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
76889a9ff000-76889aa03000 r--p 001fe000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
76889aa03000-76889aa05000 rw-p 00202000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
76889aa05000-76889aa12000 rw-p 00000000 00:00 0
76889aa06000-76889aa09000 rw-p 00000000 00:00 0
76889aa09000-76889aa92000 rw-p 00000000 00:00 0
76889aa92000-76889aa93000 r--p 00000000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76889aa93000-76889aaabe000 r-xp 00001000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76889aaabe000-76889aac0000 r--p 0002c000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76889aac0000-76889aac8000 r--p 00036000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76889aac8000-76889aac0000 rw-p 00038000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffcd8d56000-7ffcd8d77000 rw-p 00000000 00:00 0 [stack]
7ffcd8d7e000-7ffcd8d82000 r--p 00000000 00:00 0 [vvar]
7ffcd8d82000-7ffcd8d84000 r-xp 00000000 00:00 0 [vdso]
dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas1_address.c -o tugas1_address

```

○ Hasil ke 3x dijalan

```

dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas1_address.c -o tugas1_address
./tugas1_address
=== ENHANCED MEMORY LAYOUT ===

1. TEXT SEGMENT
main() address      : 0x567de0c2b398
func1() address     : 0x567de0c2b249
func2() address     : 0x567de0c2b2bb

2. DATA SEGMENT
global_var address  : 0x567de0c2e010
static_array address : 0x567de0c2e020
string_literal addr  : 0x567de0c2c008

3. HEAP SEGMENT
dynamic_array addr   : 0x567de6c462a0

4. STACK SEGMENT
main local var addr  : 0x7ffdf5c1eda24

5. RECURSIVE STACK
Recursive level 0 - stack addr: 0x7ffdf5c1eda04
Recursive level 1 - stack addr: 0x7ffdf5c1ed9d4
Recursive level 2 - stack addr: 0x7ffdf5c1ed94d
Recursive level 3 - stack addr: 0x7ffdf5c1ed974

PID: 812
Tekan ENTER untuk melihat /proc/[pid]/maps

567de0c2a000-567de0c2b000 r--p 00000000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
567de0c2b000-567de0c2c000 r-xp 00001000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
567de0c2c000-567de0c2d000 r--p 00002000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
567de0c2d000-567de0c2e000 r--p 00003000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
567de0c2e000-567de0c2f000 rw-p 00003000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
567de6c46000-567de6c67000 rw-p 00000000 00:00 0 [heap]
7dce9ba00000-7dce9ba28000 r--p 00000000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7dce9ba28000-7dce9bbb0000 r-xp 00028000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7dce9bbb0000-7dce9bbff000 r--p 001b0000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7dce9bbff000-7dce9bc03000 r--p 001fe000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7dce9bc03000-7dce9bc05000 rw-p 00202000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7dce9bc05000-7dce9bc12000 rw-p 00000000 00:00 0
7dce9bc15000-7dce9bc18000 rw-p 00000000 00:00 0
7dce9bc1f000-7dce9bc21000 rw-p 00000000 00:00 0
7dce9bc21000-7dce9bc22000 r--p 00000000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7dce9bc22000-7dce9bc4d000 r-xp 00001000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7dce9bc4d000-7dce9bc57000 r--p 0002c000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7dce9bc57000-7dce9bc59000 r--p 00036000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7dce9bc59000-7dce9bc5b000 rw-p 00038000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffdf5c1ce000-7ffdf5c1ef000 rw-p 00000000 00:00 0 [stack]
7ffdf5c1ef000-7ffdf5c1f3000 r--p 00000000 00:00 0 [vvar]
7ffdf5c1f3000-7ffdf5c1f5000 r-xp 00000000 00:00 0 [vdso]

```

○ Hasil ke 4x dijalankan

```

dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas1_address.c -o tugas1_address
./tugas1_address
=== ENHANCED MEMORY LAYOUT ===

1. TEXT SEGMENT
main() address      : 0x55e6d91e7398
func1() address     : 0x55e6d91e7249
func2() address     : 0x55e6d91e72bb

2. DATA SEGMENT
global_var address  : 0x55e6d91ea010
static_array address : 0x55e6d91ea020
string_literal addr  : 0x55e6d91e8008

3. HEAP SEGMENT
dynamic_array addr   : 0x55e702e1d2a0

4. STACK SEGMENT
main local var addr  : 0x7ffd13637914

5. RECURSIVE STACK
Recursive level 0 - stack addr: 0x7ffd136378f4
Recursive level 1 - stack addr: 0x7ffd136378c4
Recursive level 2 - stack addr: 0x7ffd13637894
Recursive level 3 - stack addr: 0x7ffd13637864

PID: 820
Tekan ENTER untuk melihat /proc/[pid]/maps

55e6d91e6000-55e6d91e7000 r--p 00000000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
55e6d91e7000-55e6d91e8000 r-xp 00001000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
55e6d91e8000-55e6d91e9000 r--p 00002000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
55e6d91e9000-55e6d91ea000 r--p 00002000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
55e6d91ea000-55e6d91eb000 rw-p 00003000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
55e702e1d000-55e702e1e000 rw-p 00000000 00:00 0 [heap]
7ca887200000-7ca887228000 r--p 00000000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ca887228000-7ca8873b0000 r-xp 00028000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ca8873b0000-7ca8873ff000 r--p 001b0000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ca8873ff000-7ca887403000 r--p 001fe000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ca887403000-7ca887405000 rw-p 00202000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
7ca887405000-7ca887412000 rw-p 00000000 00:00 0
7ca887532000-7ca887535000 rw-p 00000000 00:00 0
7ca887535000-7ca88753e000 rw-p 00000000 00:00 0
7ca88753e000-7ca88753f000 r--p 00000000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ca88753f000-7ca88756a000 r-xp 00001000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ca88756a000-7ca887574000 r--p 0002c000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ca887574000-7ca887576000 r--p 00036000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ca887576000-7ca887578000 rw-p 00038000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffd13619000-7ffd1363a000 rw-p 00000000 00:00 0 [stack]
7ffd136e000-7ffd136f2000 r--p 00000000 00:00 0 [vvar]
7ffd136f2000-7ffd136f4000 r-xp 00000000 00:00 0 [vdso]
dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas1_address.c -o tugas1_address

```

○ Hasil ke 5x dijalankan

```

dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas1_address.c -o tugas1_address
./tugas1_address
=== ENHANCED MEMORY LAYOUT ===

1. TEXT SEGMENT
main() address      : 0x5b3b26d43398
func1() address     : 0x5b3b26d43249
func2() address     : 0x5b3b26d432bb

2. DATA SEGMENT
global_var address  : 0x5b3b26d46010
static_array address : 0x5b3b26d46020
string_literal addr  : 0x5b3b26d44008

3. HEAP SEGMENT
dynamic_array addr   : 0x5b3b3b97e2a0

4. STACK SEGMENT
main local var addr  : 0x7ffdbb8dc714

5. RECURSIVE STACK
Recursive level 0 - stack addr: 0x7ffdbb8dc6f4
Recursive level 1 - stack addr: 0x7ffdbb8dc6c4
Recursive level 2 - stack addr: 0x7ffdbb8dc694
Recursive level 3 - stack addr: 0x7ffdbb8dc664

PID: 828
Tekan ENTER untuk melihat /proc/[pid]/maps

5b3b26d42000-5b3b26d43000 r--p 00000000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
5b3b26d43000-5b3b26d44000 r-xp 00001000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
5b3b26d44000-5b3b26d45000 r--p 00002000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
5b3b26d45000-5b3b26d46000 r--p 00002000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
5b3b26d46000-5b3b26d47000 rw-p 00003000 08:30 60426 /home/dea/praktikum-memory/tugas1_address
5b3b3b97e000-5b3b3b97f000 rw-p 00000000 00:00 0 [heap]
791afcc00000-791afcc28000 r--p 00000000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
791afcc28000-791afcdb0000 r-xp 00028000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
791afcdb0000-791afcdf0000 r--p 001b0000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
791afcdf0000-791afce03000 r--p 001fe000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
791afce03000-791afce05000 rw-p 00202000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
791afce05000-791afce12000 rw-p 00000000 00:00 0
791afce12000-791afce12000 rw-p 00000000 00:00 0
791afce12000-791afce12000 rw-p 00000000 00:00 0
791afce12000-791afce12000 rw-p 00000000 00:00 0
791afce12000-791afce12000 r--p 00000000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
791afce12000-791afce12000 r-xp 00001000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
791afce12000-791afce12000 r--p 0002c000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
791afce12000-791afce12000 r--p 00036000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
791afce12000-791afce12000 r--p 00038000 08:30 49554 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffdbb8db000-7ffdbb8de000 rw-p 00000000 00:00 0 [stack]
7ffdbb8de000-7ffdbb8e2000 r--p 00000000 00:00 0 [vvar]
7ffdbb8e2000-7ffdbb8e4000 r-xp 00000000 00:00 0 [vdso]
dea@LAPTOP-H54VQ765:~/praktikum-memory$ |

```

## TUGAS 2: IMPLEMENTASI MEMORY ALLOCATOR

### A. IMPLEMENTASI NEXT-FIT ALGORITHM

File: tugas2\_allocator.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define MAX_BLOCKS 20
```

```
#define MEMORY_SIZE 1000
```

```
typedef struct {
```

```
    int size;
```

```
    int allocated;
```

```
} Block;
```

```
Block memory[MAX_BLOCKS];
```

```
int block_count;
```

```
int last_pos = 0;
```

```
/* ===== INITIALIZATION ===== */
```

```
void init_memory() {
```

```
    memory[0].size = MEMORY_SIZE;
```

```
    memory[0].allocated = 0;
```

```
    block_count = 1;
```

```
}
```

```
/* ===== DISPLAY ===== */
```

```
void display_memory() {
```

```
    printf("Block\tSize\tStatus\n");
```

```
    for (int i = 0; i < block_count; i++) {
```

```
        printf("%d\t%d\t%s\n", i, memory[i].size,
```

```
               memory[i].allocated ? "Allocated" : "Free");
```

```
    }
```

```
}
```

```
/* ===== FIRST FIT ===== */
```

```
int first_fit(int size) {
```

```
    for (int i = 0; i < block_count; i++) {
```

```
        if (!memory[i].allocated && memory[i].size >= size) {
```

```
            memory[i].allocated = 1;
```

```
            return i;
```

```
        }
```

```
    }
```

```
    return -1;
```

```
}
```

```
/* ===== BEST FIT ===== */
```

```
int best_fit(int size) {
```

```
    int best = -1, min = MEMORY_SIZE + 1;
```

```

for (int i = 0; i < block_count; i++) {

    if (!memory[i].allocated && memory[i].size >= size &&

        memory[i].size < min) {

        min = memory[i].size;

        best = i;

    }

}

if (best != -1) memory[best].allocated = 1;

return best;

}

/* ===== WORST FIT ===== */

int worst_fit(int size) {

    int worst = -1, max = -1;

    for (int i = 0; i < block_count; i++) {

        if (!memory[i].allocated && memory[i].size >= size &&

            memory[i].size > max) {

            max = memory[i].size;

            worst = i;

        }

    }

    if (worst != -1) memory[worst].allocated = 1;

    return worst;

}

```

```
/* ===== NEXT FIT ===== */
```

```
int next_fit(int size) {  
    int count = 0;  
    int i = last_pos;  
    while (count < block_count) {  
        if (!memory[i].allocated && memory[i].size >= size) {  
            memory[i].allocated = 1;  
            last_pos = i;  
            return i;  
        }  
        i = (i + 1) % block_count;  
        count++;  
    }  
    return -1;  
}
```

```
/* ===== FRAGMENTATION ===== */
```

```
void fragmentation() {  
    int total_free = 0, largest = 0, holes = 0;  
    for (int i = 0; i < block_count; i++) {  
        if (!memory[i].allocated) {  
            total_free += memory[i].size;  
            holes++;  
            if (memory[i].size > largest)  
                largest = memory[i].size;  
        }  
    }  
}
```

```

    }

}

float frag = total_free ? (1 - (float)largest / total_free) * 100 : 0;

printf("Total Free: %d | Largest Hole: %d | Holes: %d | Frag: %.2f%%\n",

    total_free, largest, holes, frag);

}

```

```

int main() {

    init_memory();


    printf("First Fit allocation: %d\n", first_fit(200));

    printf("Best Fit allocation : %d\n", best_fit(150));

    printf("Worst Fit allocation: %d\n", worst_fit(100));

    printf("Next Fit allocation : %d\n", next_fit(180));


    display_memory();

    fragmentation();

    return 0;

}

```

## B. ANALISIS

1. Buat tabel perbandingan:

| Algoritma | Waktu Eksekusi (µs) | Fragmentasi (%) | Failed Allocations |
|-----------|---------------------|-----------------|--------------------|
| First-Fit | Cepat               | Sedang          | Sedikit            |
| Best-Fit  | Lambat              | Rendah          | Jarang             |
| Worst-Fit | Sedang              | Tinggi          | Lebih Sering       |
| Next-Fit  | Paling cepat        | Sedang          | Sedikit            |

2. Analisis

- Algoritma mana yang paling cepat? Mengapa?

Next-Fit, Karena tidak selalu scam dari awal dan melanjutkan pencarian dari posisi terakhir.

- Algoritma mana yang paling efisien dalam mengurangi fragmentasi?

Best-Fit, Karena Menggunakan hole terkecil yang cukup dan Sisa ruang lebih kecil.

- Dalam situasi apa setiap algoritma lebih unggul?

- First-Fit = Sistem sederhana, alokasi cepat
- Best-Fit = Sistem dengan keterbatasan memori
- Worst-Fit = Hampir tidak digunakan di sistem nyata
- Next-Fit = Sistem real-time, butuh kecepatan

3. Buat grafik visualisasi memory allocation untuk setiap algoritma

### C. HASIL OUTPUT

```
task2_allocator: command not found
dea@LAPTOP-H54VQ765:~/praktikum-memory$ nano tugas2_allocator.c
dea@LAPTOP-H54VQ765:~/praktikum-memory$ nano tugas2_allocator.c
dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas2_allocator.c -o tugas2_allocator
./tugas2_allocator
First Fit allocation: 0
Best Fit allocation : -1
Worst Fit allocation: -1
Next Fit allocation : -1
Block  Size  Status
0      1000  Allocated
Total Free: 0 | Largest Hole: 0 | Holes: 0 | Frag: 0.00%
dea@LAPTOP-H54VQ765:~/praktikum-memory$
```

### TUGAS 3: ADVANCED PAGING SYSTEM

#### A. IMPLEMENTASI PAGE REPLACEMENT – FIFO

File: tugas3\_paging.c

```
#include <stdio.h>
```

```
#define FRAMES 4
```

```
#define PAGES 20
```

```
int fifo_faults = 0, lru_faults = 0;
```

```
/* ===== FIFO ===== */
```

```
void fifo(int pages[]) {
```

```
    int frame[FRAMES], index = 0;
```

```
    for (int i = 0; i < FRAMES; i++) frame[i] = -1;
```

```
    for (int i = 0; i < PAGES; i++) {
```

```
        int found = 0;
```

```
        for (int j = 0; j < FRAMES; j++)
```

```
            if (frame[j] == pages[i]) found = 1;
```

```
        if (!found) {
```

```
            frame[index] = pages[i];
```

```
            index = (index + 1) % FRAMES;
```

```
            fifo_faults++;
```

```
        }
```

```

    }

}

/* ===== LRU ===== */

void lru(int pages[]) {

    int frame[FRAMES], recent[FRAMES];

    for (int i = 0; i < FRAMES; i++) frame[i] = -1;

    for (int i = 0; i < PAGES; i++) {

        int pos = -1;

        for (int j = 0; j < FRAMES; j++)

            if (frame[j] == pages[i]) pos = j;

        if (pos == -1) {

            int lru_index = 0;

            for (int j = 1; j < FRAMES; j++)

                if (recent[j] < recent[lru_index]) lru_index = j;

            frame[lru_index] = pages[i];

            recent[lru_index] = i;

            lru_faults++;

        } else {

            recent[pos] = i;

        }

    }

}

```

```

int main() {

    int pages[PAGES] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};

    fifo(pages);

    lru(pages);

    printf("FIFO Page Faults: %d\n", fifo_faults);

    printf("LRU Page Faults : %d\n", lru_faults);

    return 0;

}

```

## B. ANALISIS

1. Buat tabel trace untuk setiap algoritma:

- Trace Table FIFO (4 Frames)

| Step | Reference | Frame 0 | Frame 1 | Frame 2 | Frame 3 | Fault? |
|------|-----------|---------|---------|---------|---------|--------|
| 1    | 7         | 7       | -       | -       | -       | YA     |
| 2    | 0         | 7       | 0       | -       | -       | YA     |
| 3    | 1         | 7       | 0       | 1       | -       | YA     |
| 4    | 2         | 7       | 0       | 1       | 2       | YA     |
| 5    | 0         | 7       | 0       | 1       | 2       | NO     |
| 6    | 3         | 3       | 0       | 1       | 2       | YA     |

(Page fault FIFO lebih banyak)

- Trace Table LRU (4 Frames)

| Step | Ref | F0 | F1 | F2 | F3 | Fault |
|------|-----|----|----|----|----|-------|
| 1    | 7   | 7  | -  | -  | -  | YA    |
| 2    | 0   | 7  | 0  | -  | -  | YA    |
| 3    | 1   | 7  | 0  | 1  | -  | YA    |
| 4    | 2   | 7  | 0  | 1  | 2  | YA    |

|   |   |   |   |   |   |    |
|---|---|---|---|---|---|----|
| 5 | 0 | 7 | 0 | 1 | 2 | NO |
| 6 | 3 | 0 | 3 | 1 | 2 | YA |

2. Hitung Belady's Anomaly:

| Frames | FIFO Fault | LRU Fault |
|--------|------------|-----------|
| 3      | Tinggi     | Menurun   |
| 4      | Naik       | Turun     |
| 5      | Turun      | Turun     |

FIFO mengalami Belady's Anomaly sedangkan LRU tidak

3. Analisis Performa

- Algoritma mana yang lebih baik?

LRU lebih baik karena berbasis pola akses

- Kapan FIFO lebih baik dari LRU?

Pola akses sangat sederhana dan Overhead harus rendah

## C. HASIL OUTPUT

```

Total Free: 0 | Largest Node: 0 | Nodes: 0 | Frag: 0.00%
dea@LAPTOP-H54VQ765:~/praktikum-memory$ nano tugas3_paging.c
dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas3_paging.c -o tugas3_paging
./tugas3_paging
FIFO Page Faults: 10
LRU Page Faults : 12

```

## TUGAS 4: MULTI-LEVEL PAGE TABLE

### A. THREE-LEVEL PAGE TABLE

File: tugas4\_multilevel.c

```
#include <stdio.h>
```

```
#define L1 4
```

```
#define L2 4
```

```
#define L3 4
```

```
int table[L1][L2][L3];
```

```
void init() {
```

```
    for (int i=0;i<L1;i++)
```

```
        for (int j=0;j<L2;j++)
```

```
            for (int k=0;k<L3;k++)
```

```
                table[i][j][k] = -1;
```

```
}
```

```
void allocate(int l1, int l2, int l3, int frame) {
```

```
    table[l1][l2][l3] = frame;
```

```
}
```

```
void translate(int l1, int l2, int l3) {
```

```
    if (table[l1][l2][l3] != -1)
```

```
        printf("Virtual [%d,%d,%d] -> Frame %d\n",
```

```
            l1,l2,l3,table[l1][l2][l3]);
```

```
    else
```

```
        printf("PAGE FAULT\n");
```

```
}
```

```
int main() {
```

```
    init();
```

```

allocate(1,2,3,10);

translate(1,2,3);

translate(0,0,1);

return 0;

}

```

## B. ANALISIS PERBANDINGAN

### 1. Memory Overhead Comparison:

| Structure    | Entries       | Size per Entry | Total Memory | Space Efficiency |
|--------------|---------------|----------------|--------------|------------------|
| Single-Level | Banyak        | Tinggi         | 2            | Rendah           |
| Two-Level    | Sedang        | Sedang         | 3            | Baik             |
| Three-Level  | Sedikit aktif | Rendah         | 4            | Sangat baik      |
| Inverted     | =Frame        | Paling rendah  | Hash + 1     | Sangat Efisiensi |

### 2. Translation Speed:

- Berapa pengaruh TLB (Translation Lookaside Buffer)?

TLB hit 1 akses saja dan Sangat mempercepat translasi alamat

### 3. Kasus Penggunaan:

- Struktur mana yang cocok untuk sistem 32-bit vs 64-bit?

32 bit = Two level VS 64 bit = Three-Level

- Kapan inverted page table lebih efisien?

Sistem besar & banyak proses

## C. HASIL OUTPUT

```

ENO Page Faults : 12
dea@LAPTOP-H54VQ765:~/praktikum-memory$ nano tugas4_multilevel.c
dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas4_multilevel.c -o tugas4_multilevel
./tugas4_multilevel
Virtual [1,2,3] -> Frame 10
PAGE FAULT
dea@LAPTOP-H54VQ765:~/praktikum-memory$

```

## TUGAS 5: SWAPPING MECHANISM

### A. BASIC SWAPPING IMPLEMENTATION

File: tugas5\_swapping.c

```
#include <stdio.h>
```

```
#include <time.h>
```

```
typedef struct {
```

```
    int pid;
```

```
    int in_memory;
```

```
    int priority;
```

```
    time_t last_access;
```

```
} Process;
```

```
Process p[3];
```

```
void swap_out(int i) {
```

```
    p[i].in_memory = 0;
```

```
    printf("Process %d swapped OUT\n", p[i].pid);
```

```
}
```

```

void swap_in(int i) {

    p[i].in_memory = 1;

    p[i].last_access = time(NULL);

    printf("Process %d swapped IN\n", p[i].pid);

}

```

```

int main() {

    p[0]=(Process){1,1,3,time(NULL)};

    p[1]=(Process){2,1,1,time(NULL)};

    p[2]=(Process){3,1,2,time(NULL)};


    swap_out(1);

    swap_in(1);

    return 0;

}

```

## B. ANALISIS

### 1. Metrik Performa:

| <b>Strategi</b> | <b>Swap Out</b> | <b>Swap In</b> | <b>Thrashing</b> |
|-----------------|-----------------|----------------|------------------|
| FIFO            | Tinggi          | Tinggi         | Tinggi           |
| LRU             | Rendah          | Rendah         | Paling rendah    |
| Priority        | Sedang          | Sedang         | Sedang           |

### 2. Analisis

- Strategi mana yang meminimalkan thrashing?

LRU

- Bagaimana ukuran memory mempengaruhi performa?

Swapping jarang

- Kapan swapping lebih baik daripada killing process?

Proses masih aktif dan Prioritas tinggi

### C. HASIL OUTPUT

```
PAGE FAULT
dea@LAPTOP-H54VQ765:~/praktikum-memory$ nano tugas5_swapping.c
dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas5_swapping.c -o tugas5_swapping
./tugas5_swapping
Process 2 swapped OUT
Process 2 swapped IN
dea@LAPTOP-H54VQ765:~/praktikum-memory$
```

## TUGAS 6: ARCHITECTURE ANALYSIS

### A. INTEL ARCHITECTURE ANALYSIS

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    printf("CPU Info:\n");
```

```
    system("lscpu | grep Architecture");
```

```
    printf("\nPage Size:\n");
```

```
    system("getconf PAGE_SIZE");
```

```
    printf("\nMemory Info:\n");
```

```
    system("free -h");
```

```
    printf("\nMemory Map:\n");
```

```

system("cat /proc/self/maps | head");

return 0;

}

```

## B. PERBANDINGAN ARSITEKTUR

1. Bandingkan spesifikasi sistem Anda dengan:

| Arsitektur | Addres Space  |
|------------|---------------|
| IA-32      | 32-bit (4 GB) |
| x86-64     | 48-57 bit     |
| ARMv8      | 48 bit        |

2. Jelaskan:

- Berapa bit address space yang digunakan?  
64-bit
- Bagaimana implementasi multi-level paging?  
4–5 level
- Berapa ukuran maksimum virtual memory?  
x86-64  $\approx$  256 TB

## C. HASIL OUTPUT

```

dea@LAPTOP-H54VQ765:~/praktikum-memory$ nano tugas6_architecture.c
dea@LAPTOP-H54VQ765:~/praktikum-memory$ gcc tugas6_architecture.c -o tugas6_architecture
./tugas6_architecture
CPU Info:
Architecture:          x86_64

Page Size:
4096

Memory Info:

```

|       | total | used  | free  | shared | buff/cache | available |
|-------|-------|-------|-------|--------|------------|-----------|
| Mem:  | 1.7Gi | 419Mi | 1.2Gi | 3.8Mi  | 198Mi      | 1.3Gi     |
| Swap: | 1.0Gi | 0B    | 1.0Gi |        |            |           |

```

Memory Map:
5ba2abd1d000-5ba2abd1f000 r--p 00000000 08:30 49445 /usr/bin/cat
5ba2abd1f000-5ba2abd24000 r-xp 00002000 08:30 49445 /usr/bin/cat
5ba2abd24000-5ba2abd26000 r--p 00007000 08:30 49445 /usr/bin/cat
5ba2abd26000-5ba2abd27000 r--p 00008000 08:30 49445 /usr/bin/cat
5ba2abd27000-5ba2abd28000 rw-p 00009000 08:30 49445 /usr/bin/cat
5ba2e358b000-5ba2e35ac000 rw-p 00000000 00:00 0 [heap]
77ebe0a00000-77ebe0a28000 r--p 00000000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
77ebe0a28000-77ebe0bb0000 r-xp 00028000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
77ebe0bb0000-77ebe0bff000 r--p 001b0000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
77ebe0bff000-77ebe0c03000 r--p 001fe000 08:30 49664 /usr/lib/x86_64-linux-gnu/libc.so.6
dea@LAPTOP-H54VQ765:~/praktikum-memory$

```